Software Engineering Lab

Lab 32

Database Sharding & Partitioning

Skalabilitas Horizontal Melalui Routing, Partition Pruning, dan Consistent Hashing

Lukman / lukman-ss

Batas Fisik Database Single-Instance

Tiga batasan hardware memaksa transisi dari vertical scaling ke partitioning dan sharding

BATAS MESIN TUNGGAL

Batas RAM: Buffer pool cache tidak lagi menampung working dataset aktif
Batas Disk IOPS: Saturasi write throughput dan antrean fsync log WAL
Lock Contention: Transaksi konkuren berebut lock pada index tree dan page yang sama

BIAYA VERTICAL SCALING

Biaya hardware tumbuh eksponensial untuk peningkatan kapasitas marginal
Single point of failure: Kegagalan hardware host menghentikan seluruh layanan
Batas arsitektural: Bus memori dan jumlah core CPU memiliki batas fisik

DUA JALUR SKALASI

Logical Partitioning: Memecah tabel internal pada satu engine database yang sama
Physical Sharding: Membagi dataset ke beberapa independent database nodes
Kombinasi keduanya menjawab skalabilitas lifecycle dan throughput transaksi

Sistem penyimpanan monolitik menghadapi batas fisik tak terelakkan ketika volume data dan laju transaksi melonjak. Peningkatan kapasitas server tunggal menemui titik jenuh pada batasan I/O storage, memori buffer pool, dan lock contention internal. Memahami perbedaan antara pemisahan logis di dalam satu mesin dan pemisahan fisik ke kluster multi-node adalah langkah pertama menuju arsitektur data terdistribusi yang berkelanjutan.

Vertical scaling hits a hard hardware ceiling; horizontal partitioning distributes the blast radius.

Logical Partitioning vs Physical Sharding

Dua paradigma berbeda dengan trade-off operasional, isolasi, dan koordinasi yang kontras

Dimensi	Logical Partitioning	Physical Sharding
Instance Host	1 Database Server	N Node Server Terpisah
Batas Skalasi	Batas Hardware Host	Tak Terbatas (Scale-out)
Cross-Join	Native SQL Engine	Memerlukan Scatter-Gather
Distributed ID	Auto-Increment Lokal	Sequence Generator / UUIDv7

Logical partitioning membagi satu tabel besar menjadi beberapa sub-tabel di server yang sama, memungkinkan query engine memindai hanya rentang data yang relevan. Sebaliknya, physical sharding memecah dataset ke beberapa server independen yang memiliki CPU, RAM, dan disk tersendiri. Sharding menuntut koordinasi routing dan generator ID terdistribusi, tetapi membuka jalan untuk skalabilitas horizontal linear melampaui kapasitas mesin mana pun.

LOGICAL PARTITIONING

Dikelola oleh query engine database internal (PostgreSQL/MySQL)
Tabel induk logis dipecah menjadi sub-tabel fisik terpisah
Optimal untuk manajemen lifecycle data dan query pruning

PHYSICAL SHARDING

Setiap shard adalah database instance independen tanpa shared state
Membutuhkan layer router pintar untuk mengarahkan transaksi
Menghilangkan single point of failure pada level penyimpanan data

Partitioning optimizes single-node query pruning; sharding breaks single-node resource limits.

Mekanisme Range Partitioning pada Single Node

Sub-tabel independen berdasarkan rentang temporal waktu transaksi

STRUKTUR PARTISI TEMPORAL

```
sales_logs (Tabel Induk Logis)
 ├── p2026_q1: [2026-01-01 s.d. 2026-04-01)
 ├── p2026_q2: [2026-04-01 s.d. 2026-07-01)
 ├── p2026_q3: [2026-07-01 s.d. 2026-10-01)
 └── p2026_q4: [2026-10-01 s.d. 2027-01-01)
```

```
type PartitionRange struct {
    Name  string
    Start time.Time
    End   time.Time
}
type Table struct { partitions []*Partition }
```

MANFAAT OPERASIONAL

Tiap partisi memiliki index B-tree terpisah yang muat di RAM
Query berbasis tanggal secara deterministik diarahkan ke sub-tabel target
Menghindari degradasi index tree raksasa yang membengkak

Dalam implementasi range partitioning, tabel logis sales_logs dipetakan ke partisi kuartalan yang terisolasi. Setiap partisi hanya menerima record yang nilai timestamp-nya berada dalam interval waktu yang ditentukan. Pemisahan ini menjaga agar struktur index B-tree masing-masing partisi tetap kompak dan tetap berada di dalam memori cache, mencegah degradasi performa yang lazim terjadi pada index tabel tunggal berukuran miliaran baris.

Time-bounded partition ranges keep working B-tree indices compact enough to fit in RAM.

Partition Pruning: Eliminasi Pindaian Partisi

Query optimizer memotong partisi di luar rentang filter filter WHERE

ALGORITMA PRUNING

Query filter: WHERE created_at BETWEEN '2026-04-01' AND '2026-06-30'
Evaluasi rentang interval: Hanya p2026_q2 yang beririsan
Engine mengabaikan p2026_q1, p2026_q3, dan p2026_q4 secara mutlak

```
Query Range: 2026-04-01 to 2026-06-30
Partitions Scanned : 1 / 4
Partitions Pruned  : 3 / 4 (75% I/O dihemat)
Records Found      : 1 record tepat
```

KONDISI PRUNING GAGAL

Penggunaan fungsi pada kolom partisi: WHERE DATE(created_at) = ...
Klausa WHERE tanpa mencantumkan kunci partisi memicu full table scan
Tipe data yang tidak presisi memaksa implicit type cast pada runtime

Partition pruning adalah kemampuan query engine untuk mengeliminasi pemindaian sub-tabel yang tidak beririsan dengan predikat WHERE. Pada uji coba lab, rentang tanggal Q2 membuat database hanya memindai 1 dari 4 partisi yang ada, langsung memotong 75 persen pemindaian disk. Namun, pruning akan gagal total jika query membungkus kolom partisi dengan fungsi ekspresi atau jika predikat tidak menyertakan kunci partisi sama sekali.

Partition pruning turns sequential scans into targeted sub-tree lookups by checking range boundaries first.

Drop Partition Instan vs Expensive DELETE

Operasi metadata O(1) menggantikan jutaan tombstone row dan overhead VACUUM

Operasi	DELETE FROM …	DROP PARTITION
Waktu Eksekusi	O(N) Berjam-jam	O(1) Milidetik
Write Amplification	Sangat Tinggi (WAL Row)	Minimum (Metadata File)
Disk Space Reclaim	Butuh VACUUM FULL	Langsung ke File System
Lock Impact	Row Lock Berkepanjangan	Exclusive Lock Singkat

```
func (t *Table) DropPartition(name string) bool {
    t.mu.Lock(); defer t.mu.Unlock()
    // Hapus referensi metadata dan bebaskan sub-tabel
    // O(1) unlinking tanpa memindai isi baris
}
```

Menghapus data kadaluwarsa pada tabel monolitik raksasa menggunakan klausa DELETE menghasilkan jutaan catatan transaksi WAL dan tombstone, yang berujung pada table bloat dan kebutuhan VACUUM berat. Dengan table partitioning, data lama dibuang menggunakan perintah DROP PARTITION yang mengeksekusi operasi unlinking file O(1). Ruang penyimpanan langsung kembali ke sistem operasi dalam hitungan milidetik tanpa mengganggu transaksi produksi yang sedang berlangsung.

IMPLIKASI PRODUKSI

Data retensi audit/transaksi 90 hari dapat dihapus secara instan
Mencegah tabel membengkak (bloat) yang memperlambat sequential scan
Tidak membebani worker background autovacuum pada database aktif

Dropping a partition unlinks files at the filesystem layer; deleting rows burns write I/O per tombstone.

Anatomi Kluster Sharding Fisik

Arsitektur shared-nothing: Routing layer, Shard nodes, dan Metadata catalog

KOMPONEN KLUSTER

Aplikasi Klien: Mengirim query SQL tanpa perlu tahu topologi node
Routing Coordinator: Menerjemahkan Shard Key ke Shard Node target
Shard Nodes: Server PostgreSQL/MySQL independen (Shared-Nothing)
Catalog / GSI: Menyimpan pemetaan metadata dan indeks sekunder

```
type Router interface {
    GetShard(key string) (string, error)
    AddShard(shardID string)
    RemoveShard(shardID string)
}
type Cluster struct { shards map[string]*Shard }
```

SIFAT SHARED-NOTHING

Tidak ada memori atau disk yang dibagi antar server shard
Kegagalan satu node hanya berdampak pada subset data terkait
Kapasitas komputasi dan write I/O bertambah secara linear per node

Arsitektur physical sharding menganut prinsip shared-nothing di mana setiap server database beroperasi mandiri tanpa berbagi memori maupun disk storage. Lapisan koordinator routing bertindak sebagai perantara yang mengevaluasi shard key pada query untuk menemukan shard tujuan secara deterministik. Desain ini mengisolasi kegagalan node dan memastikan penambahan node baru benar-benar menambah kapasitas total sistem secara proporsional.

Shared-nothing sharding separates state completely; routers translate logical keys into physical node targets.

Write Hotspot Akibat Monotonic Shard Key

Kunci sekuensial mengalirkan 100% traffic penulisan ke satu node aktif

PENYEBAB HOTSPOT

Penggunaan timestamp (created_at) atau autoincrement ID sebagai shard key
Semua transaksi baru pada jendela waktu yang sama memiliki nilai kunci identik
Router mengarahkan seluruh traffic write ke satu server fisik yang sama

```
Ingest 1.000 transaksi berurutan dengan kunci tanggal:
  Node shard-0 :    0 records (0%)
  Node shard-1 :    0 records (0%)
  Node shard-2 : 1000 records (100% [##########])
  Node shard-3 :    0 records (0%)
Hasil: Write Hotspot Parah! Shard lain menganggur.
```

DAMPAK SISTEMIK

Satu node kehabisan disk I/O dan connection pool
Node lain di kluster mengalami idle resource pemborosan biaya
Kluster multi-node berkinerja lebih buruk daripada single-instance

Memilih kunci yang bertambah secara monotonik seperti tanggal atau auto-increment integer adalah jebakan paling berbahaya dalam arsitektur sharding. Pengujian lab menunjukkan bahwa 1.000 transaksi berturut-turut pada hari yang sama seluruhnya bermuara pada shard-2, sementara node lainnya menganggur. Alih-alih mendapatkan distribusi beban merata, sistem justru menciptakan bottleneck parah di satu server sementara sumber daya node lain terbuang sia-sia.

Sequential keys serialize writes into a single shard, turning multi-node clusters into single-node bottlenecks.

Kunci Berkardinalitas Tinggi untuk Distribusi Merata

Distribusi seimbang di seluruh node fisik dengan tenant_id atau user_id

KRITERIA KUNCI IDEAL

Kardinalitas Tinggi: Memiliki jutaan nilai distinct (user_id, account_id)
Frekuensi Rendah: Tidak ada satu entitas yang menguasai >5% transaksi
Non-Monotonik: Pola penulisan acak di sepanjang ruang domain kunci

```
Ingest 1.000 data menggunakan kunci tenant_id:
  Node shard-0 : 400 records (40%)
  Node shard-2 : 200 records (20%)
  Node shard-3 : 400 records (40%)
Distribusi menyebar ke seluruh node fisik secara proporsional.
```

HASHED SHARDING STRATEGY

Jika entitas bisnis monotonik, bungkus dengan fungsi hash (MD5, Murmur, FNV)
Hash mengacak urutan temporal menjadi distribusi pseudorandom seragam
Kelemahan: Query berbasis range pada kunci asli tidak lagi terlokalisasi

Kunci sharding yang baik wajib memiliki kardinalitas tinggi dan distribusi nilai acak, seperti user_id atau tenant_id. Dengan mendistribusikan data berdasarkan identifier ini, beban penulisan tersebar merata di seluruh node kluster. Jika aplikasi terpaksa menggunakan kunci berurutan, pendekatan hashed sharding diterapkan untuk mengacak nilai kunci melalui fungsi kriptografis atau hash non-kriptografis sebelum menentukan nomor shard tujuan.

High-cardinality keys spread transactions uniformly; hashing scrambles temporal patterns into random scatter.

Routing Hash Modulo: Rumus Naif & Sederhana

Pemetaan langsung hash(key) % N: Cepat dihitung, rapuh saat cluster resize

RUMUS MATEMATIS

ShardID = hash(key) % N
Di mana N adalah jumlah node fisik aktif di kluster
Contoh: hash('tenant_12') = 982312, 982312 % 4 = Shard 0

```
func (m *ModuloRouter) GetShard(key string) (string, error) {
    m.mu.RLock(); defer m.mu.RUnlock()
    h := hashKey(key)
    idx := int(h % uint64(len(m.shards)))
    return m.shards[idx], nil
}
```

KELEBIHAN & KETERBATASAN

Kelebihan: Waktu pencarian O(1) konstan tanpa struktur data tambahan
Kelebihan: Distribusi kunci sangat merata selama hash function baik
Kelemahan: Mengubah nilai N merusak pemetaan hampir seluruh data di kluster

Metode Hash Modulo membagi nilai hash kunci dengan jumlah total node N menggunakan operator sisa bagi. Pendekatan ini sangat sederhana dan menghasilkan lookup berkecepatan O(1) tanpa memori tambahan. Namun, kelemahannya terletak pada keterikatan nilai N dalam rumus; saat satu node ditambahkan atau dikurangi, hampir semua kunci lama akan menghasilkan indeks baru yang berbeda dari lokasi fisiknya saat ini.

Hash modulo delivers instant O(1) routing, but ties node count directly into the key-to-node mapping.

Bencana Resharding Hash Modulo

Menambah satu node memicu relokasi masif ~80% data di seluruh sistem

PERSENTASE TEORITIS RELOKASI

Rasio data yang berpindah saat scale-out dari N ke N+1:
Moved Ratio = N / (N + 1)
Dari 4 node ke 5 node = 4/5 = 80.00% kunci wajib dipindahkan!

```
Cluster Resize: 4 Shards -> 5 Shards
Keys Remapped   : 7.984 / 10.000 keys
Persentase Pindah: 79.84% (Sangat akurat dengan teori 80%)
Kunci yang bertahan di tempat: Hanya 20.16%
```

KONSEKUENSI PRODUKSI

Saturasi bandwidth jaringan antar database untuk transfer data
Periode read-only atau locking berkepanjangan pada aplikasi
Tingkat risiko kegagalan migrasi dan inkonsistensi data sangat tinggi

Kelemahan fatal Hash Modulo terungkap saat kluster melakukan scale-out. Secara teoritis, menambah node dari 4 menjadi 5 akan memindahkan N/(N+1) atau 80% data. Hasil uji coba lab dengan 10.000 record membuktikan bahwa 7.984 kunci (79,84%) berubah shard. Di lingkungan produksi, pemindahan empat per lima data ini menimbulkan beban jaringan masif, lonjakan latensi, dan risiko downtime yang tidak dapat diterima.

Scaling hash modulo clusters reshuffles N/(N+1) of all existing data, saturating networks during rebalance.

Consistent Hashing: Memutus Ketergantungan N

Ruang hash sirkular melingkar 64–bit membatasi relokasi data hanya pada 1/N

KONSEP RING SIRKULAR

Ruang kunci dipetakan ke lingkaran angka 0 hingga 2^64 - 1
Setiap node fisik ditempatkan pada titik tertentu di sepanjang cincin
Kunci data berjalan searah jarum jam hingga bertemu node pertama

```
func (ch *ConsistentHashRouter) GetShard(key string) (string, error) {
    h := hashKey(key)
    idx := sort.Search(len(ch.ring), func(i int) bool {
        return ch.ring[i].hash >= h
    })
    if idx == len(ch.ring) { idx = 0 }
    return ch.ring[idx].shardID, nil
}
```

MINIMAL RELOKASI DATA

Saat node baru disisipkan, ia hanya mengambil data dari tetangga terdekat
Hanya ~1/N data yang berpindah lokasi secara keseluruhan
Node lain yang tidak bertetangga sama sekali tidak terpengaruh

Consistent Hashing memecahkan masalah redistribusi data dengan memetakan node dan kunci data ke dalam cincin sirkular berukuran 64–bit. Kunci diarahkan ke node terdekat searah jarum jam menggunakan algoritma pencarian biner sort.Search. Saat node baru ditambahkan, node tersebut hanya mengambil sebagian beban dari node tetangga terdekatnya di cincin, menjaga agar sisa data di node lain tetap utuh tanpa perpindahan.

Consistent hashing places nodes and keys on a circular ring; new nodes only capture keys from immediate neighbors.

Virtual Nodes: Mengeliminasi Hash Ring Skew

Memetakan satu node fisik ke ratusan titik token acak di cincin hash

MASALAH HASH SKEW

Pada jumlah node sedikit (misal 3–4 node), titik acak tidak menyebar rata
Satu node bisa menguasai 70% bentang cincin sementara yang lain hanya 10%
Menimbulkan ketimpangan beban penyimpanan dan throughput yang tajam

MEKANISME VIRTUAL NODES

Setiap node fisik dialokasikan R virtual token (misal 100–150 vnodes)
Format token vkey: shardID + '#' + strconv.Itoa(i)
Titik-titik virtual terdistribusi acak merata di sekeliling cincin

```
for i := 0; i < ch.vnodeCount; i++ {
    vkey := shardID + "#" + strconv.Itoa(i)
    h := hashKey(vkey)
    ch.ring = append(ch.ring, vnode{hash: h, shardID: shardID})
}
sort.Slice(ch.ring, func(i, j int) bool { ... })
```

Jika setiap server hanya memiliki satu titik token di cincin hash, posisi acak sering kali memicu ketimpangan distribusi (hash skew). Menggunakan virtual nodes (vnodes) mengatasi hal ini dengan memberikan 100 hingga 150 token virtual bagi setiap server fisik. Titik-titik ini terselip secara berselang-seling di sepanjang cincin, memastikan probabilitas penempatan kunci terbagi seimbang dan seragam ke seluruh server fisik.

Virtual nodes interleave physical servers across the ring, statistically guaranteeing balanced distribution.

Pembuktian Resharding: Modulo vs Consistent Hash

Verifikasi matematis pergeseran kunci saat kluster scale-out dari 4 ke 5 node

Metrik	Hash Modulo	Consistent Hashing (100 vnodes)
Dataset Diuji	10.000 Kunci Akun	10.000 Kunci Akun
Kunci Berpindah	7.984 Kunci	1.200 Kunci
Rasio Relokasi	79.84% Data Pindah	12.00% Data Pindah
Target Minimal	~80.00%	~20.00% (1/N teoritis)

```
// tests/sharding_test.go:111
if modMoveRatio < 0.65 { t.Errorf(...) }
// Modulo terbukti memindahkan 75.12% – 79.84% kunci
if chMoveRatio <= 0.05 || chMoveRatio > 0.40 { t.Errorf(...) }
// Consistent Hash terbukti memindahkan hanya 12.00% – 16.00% kunci
```

Uji coba otomatis di tests/sharding_test.go menguji langsung perbedaan kedua pendekatan saat menambah kapasitas dari 4 menjadi 5 node. Modulo memindahkan 7.984 dari 10.000 kunci (79,84%), sedangkan Consistent Hashing hanya memindahkan 1.200 kunci (12,00%). Hasil ini membuktikan Consistent Hashing mengeliminasi hingga 85 persen kebutuhan migrasi data, menjadikan proses penambahan node di sistem produksi jauh lebih efisien.

EFISIENSI MIGRASI DATA

Consistent Hashing menghemat hingga 85% bandwidth transfer resharding
Mereduksi durasi migrasi kluster dari jam menjadi hitungan menit
Menjaga ketersediaan aplikasi selama proses rebalancing berlangsung

Consistent hashing reduces data migration during scale-out from 80% down to 12%, saving 85% network I/O.

Dilema Query Tanpa Menyertakan Shard Key

Ketika pencarian dilakukan berdasarkan kolom sekunder seperti email atau status

SKENARIO KASUS NYATA

Tabel data dishard menggunakan tenant_id sebagai shard key utama
Alur Login User: Pengguna login hanya memasukkan email dan password
Aplikasi tidak mengetahui tenant_id pengguna sebelum membaca datanya

DUA JALUR PENYELESAIAN

Scatter-Gather: Broadcast query paralel ke seluruh shard di kluster
Global Secondary Index (GSI): Menjaga indeks global pemetaan email -> tenant_id
Masing-masing memiliki trade-off performa, biaya, dan konsistensi

RISIKO OPERASIONAL

Broadcast ke puluhan shard menghabiskan connection pool kluster
Tail latency query ditentukan oleh shard terlambat (straggler problem)
GSI memerlukan penulisan ganda (double-write) dan mekanisme sinkronisasi

Dalam database ter-shard, query yang menyertakan shard key dapat langsung diarahkan ke satu node target. Masalah muncul ketika aplikasi harus mencari data berdasarkan kolom sekunder, seperti mencari pengguna berdasarkan alamat email pada tabel yang di-shard berdasarkan tenant_id. Tanpa mengetahui shard key, query engine dihadapkan pada dua opsi: mem-broadcast query ke setiap shard (scatter-gather) atau memanfaatkan Global Secondary Index.

Queries lacking the sharding key face a hard trade-off: scatter-gather broadcast storm vs GSI lookup overhead.

Scatter-Gather: Eksekusi Broadcast Paralel

Membagi query ke semua shard secara paralel dan menggabungkan hasilnya

ALUR KERJA SCATTER-GATHER

1. Router menduplikasi query dan mengirimkannya ke N shard serentak
2. Shard memproses query lokal dan mengembalikan hasilnya via channel
3. Router menggabungkan (gather), mengurutkan, dan membatasi (LIMIT) hasil

```
for _, s := range shards {
    go func(shard *Shard) {
        select {
        case <-ctx.Done(): return
        default: ch <- shardResult{records: searchLocal(shard)}
        }
    }(s)
}
```

THE STRAGGLER PROBLEM

Respon total dibatasi oleh node yang paling lambat merespon (P99 tail)
Jika 1 dari 50 shard mengalami disk stall, seluruh query ikut tertahan
Membengkakkan CPU dan memory overhead pada router agregator

Scatter-gather mengatasi query tanpa shard key dengan menembakkan query paralel ke seluruh node shard menggunakan goroutine, lalu mengumpulkan hasilnya di koordinator. Meskipun fleksibel tanpa membutuhkan tabel indeks tambahan, pendekatan ini sangat rentan terhadap straggler problem. Latensi total kueri ditentukan oleh node paling lambat, dan overhead broadcast meningkat drastis seiring bertambahnya jumlah shard.

Scatter-gather latency is bound by the slowest responding shard in the broadcast fan-out.

Global Secondary Index: Point Lookup Terarah

Menghindari broadcast badai dengan pemetaan kunci sekunder ke shard key

STRUKTUR PEMETAAN GSI

Tabel Lookup Global (Lookup Vindex):
email: 'user_342@company.com' -> shardKey: 'tenant-42'
Point lookup tahap 1: Query GSI untuk mendapatkan shardKey
Point lookup tahap 2: Query langsung ke node target (1 node saja)

```
func (c *Cluster) GetByEmailUsingGSI(email, id string) (Record, error) {
    shardKey, ok := c.gsi.Lookup(email)
    if !ok { return Record{}, errors.New("not found") }
    return c.GetByShardKey(shardKey, id)
}
```

TRADE-OFF DOUBLE WRITE

Write Amplification: Setiap insert/update email wajib memperbarui tabel GSI
Konsistensi Data: Membutuhkan 2PC atau eventual consistency asinkron
Kapasitas GSI: Tabel GSI itu sendiri pada akhirnya harus di-shard

Global Secondary Index (GSI), yang dalam teknologi seperti Vitess dikenal sebagai Lookup Vindex, memetakan atribut sekunder langsung ke shard key utama. Query login cukup membaca indeks GSI untuk mengetahui nomor shard pemilik data, lalu melakukan point-lookup ke node bersangkutan. Keuntungan latensi baca yang drastis ini diimbangi biaya penulisan ganda (double-write) dan kebutuhan koordinasi konsistensi saat data dimutasi.

GSI trades write amplification for point-lookup speed, replacing cluster broadcasts with directed two-step reads.

Hasil Uji Coba: Broadcast vs GSI

Pengukuran aktual eksekusi query non-shard key pada kluster 4 node

```
1. Scatter-Gather Broadcast (by Email):
   Nodes Broadcasted : 4 / 4 Shards
   Records Matched   : 1
   Execution Time    : 72.834 µs
2. Global Secondary Index Lookup:
   Nodes Broadcasted : 1 (Point Lookup Terarah)
   Execution Time    : 1.208 µs (60x Lebih Cepat!)
```

Eksekusi demo lab memperlihatkan perbedaan performa nyata antara kedua strategi. Scatter-gather mem-broadcast query ke seluruh 4 shard dan selesai dalam waktu 72,83 µs, sementara GSI menyelesaikan point-lookup hanya dalam 1,20 µs—60 kali lebih cepat. Di dunia nyata dengan latensi jaringan dan koneksi I/O fisik, eliminasi broadcast ini krusial untuk mencegah kehabisan connection pool pada jam sibuk.

ANALISIS LATENSI PRODUKSI

Pada in-memory test: Selisih latensi mencapai 60 kali lipat
Pada database fisik dengan jaringan: Perbedaan menjadi 1 RTT vs N RTT
Scatter-gather menghabiskan thread pool dan memory buffer di seluruh node

KESIMPULAN DESAIN

Gunakan GSI untuk akses data frekuensi tinggi (misal: otentikasi login)
Gunakan Scatter-Gather hanya untuk laporan analitik batch frekuensi rendah
GSI mutlak diperlukan untuk mencegah connection starvation di skala besar

GSI point-lookups bypass cluster fan-out entirely, cutting network round trips and query latency by orders of magnitude.

Masalah Kunci Primer pada Multi-Shard

Auto-increment lokal menghasilkan ID duplikat di berbagai node independen

ANATOMI TABRAKAN ID

> Shard A menghasilkan ID baru: 101, 102, 103
> Shard B menghasilkan ID baru: 101, 102, 103 (Duplikasi!)
> Primary key global tidak lagi unik di level kluster

KELEMAHAN UUIDV4 ACAK

UUIDv4 (128-bit acak murni) menyelesaikan masalah collision unik
Namun keacakan bit merusak index B-tree database fisik
Memicu random page split, cache thrashing, dan fragmentasi disk parah

PERSYARATAN ID TERDISTRIBUSI

Unik global tanpa perlu koordinasi jaringan berulang
Monotonik terurut waktu (time-ordered) untuk menjaga efisiensi B-tree
Mendukung throughput tinggi hingga jutaan ID per detik

Mekanisme auto-increment bawaan database hanya menjamin keunikan di dalam satu mesin. Ketika dataset dipecah ke beberapa shard independen, masing-masing node akan menghasilkan ID lokal yang bertabrakan. Menggantinya dengan UUIDv4 memang menjamin keunikan global, tetapi sifat acaknya menghancurkan struktur indeks B-tree database fisik karena memicu page split terus-menerus dan menghancurkan cache locality.

Database auto-increment breaks across independent shards; random UUIDs degrade B-tree write performance through cache thrashing.

Strategi Distributed ID: UUIDv7 vs Block Allocator

Dua pola standar industri untuk primary key terdistribusi berkinerja tinggi

RFC 9562 UUIDV7 (TIME-ORDERED)

48-bit Unix timestamp millisecond + version 7 + 74-bit random entropy
Terurut waktu secara leksikografis (u1 < u2) — ramah append pada B-tree
Dihasilkan sepenuhnya lokal di memori aplikasi tanpa koordinasi jaringan

SEQUENCE BLOCK ALLOCATOR (VITESS)

Layanan pusat mengalokasikan blok ID numerik (misal: 1.000 ID sekaligus)
Node aplikasi mengonsumsi ID secara lokal hingga blok habis sebelum alokasi baru
Menghasilkan integer 64-bit padat dan hemat ukuran memori

```
UUIDv7 Tergenerate : 01a13905-fefc-7db1-adc7-ed5774231cd0
Sequence Block IDs : 1 2 3 4 5 6 7 8 (Block Size=5)
Keduanya terbukti bebas tabrakan pada pengujian konkuren.
```

Untuk mengatasi collision ID tanpa merusak B-tree, dua solusi diterapkan. Standar RFC 9562 UUIDv7 menggabungkan timestamp 48-bit dengan entropi acak, menghasilkan ID 128-bit yang terurut leksikografis tanpa memerlukan koordinasi jaringan. Pendekatan kedua, Sequence Block Allocation (gaya Vitess), mengambil blok nomor integer berurutan dari server pusat dalam batch besar, menghasilkan ID integer 64-bit yang padat dan efisien.

UUIDv7 provides zero-coordination time-ordered 128-bit keys; Sequence block allocation provides compact 64-bit integers with batched coordination.

Kompleksitas Transaksi Lintas Shard & TwoPC

Ketika atomisitas menuntut koordinasi antar server: Two-Phase Commit dan Fractured Reads

TWO-PHASE COMMIT (2PC)

Phase 1 (Prepare): Koordinator meminta vote commit ke semua shard terkait
Phase 2 (Commit): Jika semua setuju, koordinator menerbitkan perintah commit
Penalti: Menambah latensi round-trip dan menahan lock baris lebih lama

BAHAYA FRACTURED READS

2PC hanya menjamin atomisitas eksekusi (All or Nothing)
2PC TIDAK menyediakan isolasi transaksi lintas shard secara otomatis
Query baca konkuren dapat melihat shard A sudah commit tetapi shard B belum

ATURAN ARSITEKTUR UTAMA

Desain skema agar 95%+ transaksi terjadi di dalam 1 shard yang sama
Gunakan Co-location (Entity Grouping) berdasarkan Shard Key yang identik
Gunakan Saga Pattern atau Eventual Consistency untuk transaksi lintas shard

Ketika satu transaksi harus memperbarui data di dua shard berbeda, arsitektur terdistribusi membutuhkan protokol Two-Phase Commit (2PC). Meskipun 2PC menjamin atomisitas penulisan, protokol ini melipatgandakan latensi dan tidak memberikan isolasi baca lintas shard (rawan fractured reads). Strategi terbaik adalah mengelompokkan data terkait dalam satu shard key yang sama (co-location) sehingga transaksi terdistribusi dapat dihindari sepenuhnya.

Two-Phase Commit guarantees cross-shard atomicity, not global isolation; co-locate related records to avoid distributed transactions entirely.

Batas Model dan Ruang Lingkup Lab 32

Simulasi in-memory terfokus pada kebenaran algoritma dan invariant matematis

TERSEDIA DI LAB INI

Implementasi Range Partitioning dengan verifikasi pruning dan fast drop
Modulo Router vs Consistent Hash Router dengan 100–150 virtual nodes
Scatter-Gather Broadcast paralel dengan konteks pembatalan di Go
Global Secondary Index (GSI) dan dua generator ID terdistribusi

BELUM DICAKUP / BATASAN

Penyimpanan berbasis disk nyata (menggunakan Go map in-memory terlindungi mutex)
Mesin Two-Phase Commit (2PC) transaksi lintas shard fisik
Mekanisme data replication (Raft/Paxos) antar shard replicas
Proses online data streaming resharding tanpa downtime produksi

TUJUAN EDUKASI

Membuktikan secara deterministik angka pergeseran kunci dan invariant routing
Menghubungkan teori buku sistem terdistribusi dengan baris kode Go yang dapat di-debug

Lab ini dirancang sebagai model simulasi in-memory yang ketat untuk menguji invariant algoritma sharding dan partitioning di level kode Go. Implementasi ini tidak menyertakan penyimpanan file fisik, replikasi konsensus Raft, atau engine transaksi 2PC terdistribusi. Fokus utama lab adalah membuktikan secara kuantitatif efisiensi pruning, stabilitas relokasi consistent hashing, perbandingan kecepatan GSI, serta karakteristik ID terdistribusi.

This lab models core distributed algorithms in-memory to prove routing invariants without disk storage noise.

Checklist Keputusan Sharding vs Partitioning

Kerangka kerja pengambilan keputusan sebelum memecah arsitektur database Anda

CHECKLIST EVALUASI SISTEM

[ ] Apakah volume data melampaui kapasitas 1 server? Jika belum, gunakan Partitioning.
[ ] Apakah bottleneck pada lifecycle data historis? Gunakan Range Partitioning & Drop.
[ ] Shard Key memiliki kardinalitas tinggi dan terdistribusi acak (hindari tanggal murni).
[ ] Gunakan Consistent Hashing dengan 100+ vnodes untuk meminimalkan data shuffle.
[ ] Pasang GSI untuk pencarian atribut sekunder frekuensi tinggi.
[ ] Gunakan RFC 9562 UUIDv7 untuk primary key terdistribusi ramah B-tree.
[ ] Rancang skema co-location agar 95%+ query tidak membutuhkan cross-shard join.

KAPAN PARTITIONING CUKUP

Dataset besar tetapi I/O server masih mencukupi
Query membutuhkan partition pruning berbasis tanggal/kategori
Retensi data membutuhkan penghapusan tabel lama secara rutin

KAPAN HARUS SHARDING

Sebelum melangkah ke kompleksitas sharding fisik, tim rekayasa wajib mengevaluasi apakah logical table partitioning pada server tunggal sudah cukup untuk mengatasi masalah performa dan retensi data. Sharding fisik adalah senjata pamungkas ketika batasan I/O dan memori mesin tunggal benar-benar telah terlampaui. Ikuti checklist ini untuk memastikan pemilihan shard key, algoritma routing, dan strategi indeks sekunder Anda dirancang secara tepat sejak hari pertama.

Exhaust single-node table partitioning before adopting physical sharding; sharding is a last-resort scalability lever.

Ukuran dataset melebihi kapasitas storage fisik server terbesar
Kebutuhan ketersediaan tinggi di berbagai zona ketersediaan geografis

Eksperimen kecil, interpretasi yang teliti.

Logical partitioning prunes scans locally; consistent sharding scales writes globally without resharding shock.

Software Engineering Lab 32 – Database Sharding & Partitioning

internal/partitioning • internal/sharding
internal/idgen • tests/sharding_test.go
cmd/demo/main.go • README.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/32-database-sharding-and-partitioning`

Lukman / lukman-ss

Sumber commit: cfba3b76
Test passing 5/5 dengan race detector lolos tanpa data race
Consistent hash: 12% relokasi vs Modulo: 79.84% relokasi
GSI lookup: 1.20 µs vs Scatter-gather: 72.83 µs (60× lebih cepat)
GitHub: github.com/lukman-ss/software-engineering-lab/tree/main/labs/32-database-sharding-and-partitioning
Jalankan demo : go run ./cmd/demo
Jalankan test : go test -v -race ./...